

CSE 421: Lab 2 - Task 3

Introduction to network simulator

Introduction:

In networking research and protocol testing, simulating a network is one of the core tasks. We can try out existing networking protocols and measure their performance. Also, we can devise our own modified protocol and test its performance against the existing ones. In this lab, we will also get familiar with an open-source network simulator, ns3. We will learn how to install the simulator and run a basic simulation.

Objectives:

Getting familiar with network simulation

1. Installing and running ns3
2. Understanding a simple simulation script
3. Visualizing our simulation
4. Collecting network traces and observing the events

Task: Using NS-3 [Do this task in Ubuntu]

1. Download NS3
 - a. Go to [ns-3.43 | ns-3](#) and download the exact version of ns3
 - i. If the above link does not work, try from [here](#)
 - b. Extract the contents.
2. Install the necessary packages
 - a. **sudo apt install python3 g++ cmake ninja-build git**
 - i. **If you find any error giving this command, update and upgrade your apps first and then try the above command. To update and upgrade give command**
 - **sudo apt update**
 - **sudo apt upgrade**
 - **sudo apt install g++ python3 cmake ninja-build git**
3. Configure ns3 with c++ support
 - a. **First, go to the ns-allinone-3.43/ns-3.43 directory in the terminal and then run the command.**
 - i. **./ns3 configure --enable-examples --enable-tests**
4. Build ns3
 - a. **./ns3 build**
5. Copy the required file to the scratch folder
 - a. Go to **ns-allinone-3.43/ns-3.43/examples/tutorial** folder
 - b. Copy the [first.cc](#) file to **ns-allinone-3.43/ns-3.43/scratch** folder
6. Run your first script

- a. `./ns3 shell`
- b. `./ns3 run scratch/first.cc`

7. Try to understand the first simulation script. Go through [this PDF](#). This is the main thing about this lab. You have to understand the code and you will be asked questions from here in your viva.

8. Observing the output:

```
At time +2s client sent 1024 bytes to 10.1.1.2 port 9
At time +2.00369s server received 1024 bytes from 10.1.1.1 port 49153
At time +2.00369s server sent 1024 bytes to 10.1.1.1 port 49153
At time +2.00737s client received 1024 bytes from 10.1.1.2 port 9
```

Here we can see the traces of the network events that are performed by the server and client. We can see that 1024 bytes of data are being sent and received. Also, port 9 is used by the server as we had configured. We can also see the time in the simulation when the events happened. The transmission started at 2s because we configured the client to start at 2s.

9. Visualizing the simulation:

a. Installing NetAnim:

NetAnim is a simulation visualization tool that comes bundled with ns3. To use NetAnim, we need to perform some steps.

i. **Install prerequisite packages by running the following command:**

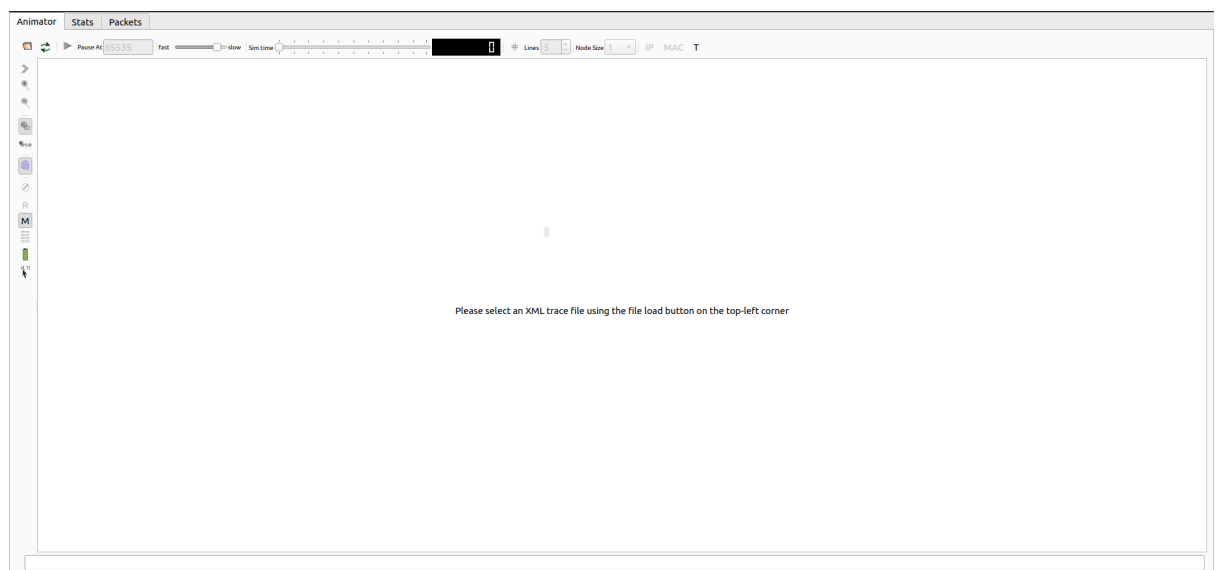
- `sudo apt install qtbase5-dev qtchooser qt5-qmake qtbase5-dev-tools`

ii. Build NetAnim:

- Move inside the **netanim-3.109** directory.
- Run the following commands:
 - a. **`qmake NetAnim.pro`**
 - b. **`make`**

iii. Run NetAnim:

- **`./NetAnim`**
- The interface should look like the following:



- b. Generating the visualization script:
 - i. Add the following line of code **before calling** the Run() method:
 - **#include "ns3/netanim-module.h"**
 - **AnimationInterface anim("first.xml");**
 - ii. This will generate a "first.xml" file after running the simulation. This file contains the necessary scripts to run the visualization of our simulation.
- c. Open the "first.xml" file in NetAnim and play the simulation. You will see a visual representation of the network events that happened in our simulation.

10. Collecting network trace:

We can collect packet captures from all our nodes in the simulation. These packet captures can help to understand the network events and troubleshoot the network.

- a. Add the following line of code **before calling** the Run() method:
 - i. **pointToPoint.EnablePcapAll("first");**
- b. This will generate a pcap file for every node in our simulation. As we had only two nodes in our simulation, we will get two pcap files after running the simulation.
- c. Open the pcap files and observe the packets.
 - i. For this we have to install Wireshark. So give command
 - **sudo apt install wireshark**
 - ii. Then double-click on the pcap files.

11. Getting network metrics and flow statistics:

- a. Add the following code block instead of calling the Run() method:

```
#include "ns3/flow-monitor-module.h"
//include the above line at the top of the code

// 1. Setup FlowMonitor
FlowMonitorHelper flowHelper;
Ptr<FlowMonitor> monitor = flowHelper.InstallAll();

// 2. Run Simulation
Simulator::Stop(Seconds(20.0));
Simulator::Run();

// 3. Process Statistics
monitor->CheckForLostPackets();
Ptr<Ipv4FlowClassifier> classifier =
DynamicCast<Ipv4FlowClassifier>(flowHelper.GetClassifier());
std::map<FlowId, FlowMonitor::FlowStats> stats = monitor->GetFlowStats();

for (auto const& [id, stat] : stats) {
    Ipv4FlowClassifier::FiveTuple t = classifier->FindFlow(id);

    std::string proto = (t.protocol == 6) ? "TCP" : (t.protocol == 17) ? "UDP" : "Unknown";

    std::cout << "FlowID: " << id << " (" << proto << " "
```

```

        << t.sourceAddress << "/" << t.sourcePort << " --> "
        << t.destinationAddress << "/" << t.destinationPort << ")\n";

std::cout << " Tx Bytes: " << stat.txBytes << "\n";
std::cout << " Rx Bytes: " << stat.rxBytes << "\n";

if (stat.rxPackets > 0) {
    std::cout << " Mean Delay: " << stat.delaySum.GetSeconds() / stat.rxPackets << " s\n";
    std::cout << " Throughput: " << (stat.rxBytes * 8.0) / 18.0 << " bps\n";
}
}

```

- b. Run the simulation and observe the output.

Home Task:

Now **vary the packet size and observe the metrics. The packet size will be derived from your student ID.** Divide your student ID in 2 equal halves. Then, first use the halves as they are and then reverse the digits of each halves. Thus you will get 4 packet sizes.

Suppose your student ID is **24252627**. After dividing into two halves, we get 2425 and 2627.

So the packet sizes are:

1. 2425
2. 2627
3. 5242 (reverse of 2425)
4. 7262 (reverse of 2627)

Collect the throughputs for each of them. Plot the Throughput vs Packet Size in this case and explain the graph.